Name Mayuri Hirade
Roll No 13163

In [28]:
```
import nltk
```

In [29]:
```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\System21\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\System21\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\System21\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\System21\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
```

Out[29]:
```
True
```

In [30]:
```
text= "Tokenization is the first step in text analytics. The process of breaking down a text paragraph into sma
```

In [5]:
```
#Sentence Tokenization
from nltk.tokenize import sent_tokenize
tokenized_text= sent_tokenize(text)
print(tokenized_text)
#Word Tokenization
from nltk.tokenize import word_tokenize
tokenized_word=word_tokenize(text)
print(tokenized_word)
```

```
['Tokenization is the first step in text analytics.', 'The process of breaking down a text paragraph into smalle
r chunks such as words or sentences is called tokenization']
['Tokenization', 'is', 'the', 'first', 'step', 'in', 'text', 'analytics', '.', 'The', 'process', 'of', 'breaking
', 'down', 'a', 'text', 'paragraph', 'into', 'smaller', 'chunks', 'such', 'as', 'words', 'or', 'sentences', 'is'
, 'called', 'tokenization']
```

In [6]:
```
# print stop words of English
import re
from nltk.corpus import stopwords
stop_words=set(stopwords.words("english"))
print(stop_words)
text= "How to remove stop words with NLTK library in Python?"
text= re.sub('[^a-zA-Z]', ' ',text)
tokens = word_tokenize(text.lower())
filtered_text=[]
for w in tokens:
     if w not in stop_words:
           filtered_text.append(w)
print("Tokenized Sentence:",tokens)
print("Filterd Sentence:",filtered_text)
```

```
{'here', 'under', 'y', "you'll", 'don', 'on', 'those', "i'll", 'was', 'too', 'of', "you'd", "aren't", 'when', 'i
n', 'what', 'shan', 'such', 'where', "shouldn't", 'has', 'once', 'which', 'by', "i'm", "that'll", 'at', 'couldn'
, 'above', 'your', 'so', 'both', "they'd", 'me', 'our', 'or', 'before', 'about', 'needn', "isn't", 'd', 'these',
'wouldn', 'won', 'yourselves', 'if', 'below', 'more', 'having', "doesn't", 'hadn', "couldn't", 'how', "he'd", 'o
ut', 'each', 've', "hasn't", "haven't", 'their', "i'd", 'against', "shan't", 'that', 'ain', 'most', 'we', "she's
", "we'll", 'didn', "it'll", 'through', 'a', 'up', 'its', 'themselves', 'were', 'not', "wouldn't", 'have', 'only
', 'theirs', 'because', 'doing', 'why', 'ourselves', 'does', 'been', 'during', 'shouldn', 'and', 'haven', 'but',
'do', 'his', 'an', 'am', "you're", 'hasn', 'is', 'very', "wasn't", 'm', 'yourself', 'them', 'same', 'will', "had
n't", 'can', 'herself', 'should', "you've", 'some', 'aren', 'further', 's', 'from', 'nor', 'this', 'o', "we've",
'my', 'down', 'itself', "we'd", 'yours', 'mightn', "i've", 'no', "mightn't", 'as', "she'll", "she'd", "weren't",
"should've", 'her', 'other', 'to', 'did', "mustn't", "needn't", 'she', 'again', 'ours', 'than', 'weren', 'off',
'wasn', 'being', 'myself', "they've", 'you', 'all', 'now', "didn't", 'are', "they'll", 'him', 'himself', "don't"
, 'own', 'with', 'll', "he'll", 'ma', 't', 'until', "won't", "we're", 'who', 'doesn', 'into', 'after', 'hers', '
isn', 'it', 'he', 'while', 'just', 'i', 'they', 'be', 'for', "he's", 'there', 'between', "they're", 'over', 'who
m', 'few', 'any', 'the', "it's", 'had', 'mustn', 'then', 're', "it'd"}
Tokenized Sentence: ['how', 'to', 'remove', 'stop', 'words', 'with', 'nltk', 'library', 'in', 'python']
Filterd Sentence: ['remove', 'stop', 'words', 'nltk', 'library', 'python']
```

In [7]:
```
from nltk.stem import PorterStemmer
e_words= ["wait", "waiting", "waited", "waits"]
ps =PorterStemmer()
for w in e_words:
     rootWord=ps.stem(w)
```

```
print(rootWord)
```

```
wait
```

In [8]:
```
from nltk.stem import WordNetLemmatizer
wordnet_lemmatizer = WordNetLemmatizer()
text = "studies studying cries cry"
tokenization = nltk.word_tokenize(text)
for w in tokenization:
      print("Lemma for {} is {}".format(w,wordnet_lemmatizer.lemmatize(w)))
```

```
Lemma for studies is study
Lemma for studying is studying
Lemma for cries is cry
Lemma for cry is cry
```

In [9]:
```
import nltk
from nltk.tokenize import word_tokenize
data="The pink sweater fit her perfectly"
words=word_tokenize(data)
for word in words:
     print(nltk.pos_tag([word]))
```

```
[('The', 'DT')]
[('pink', 'NN')]
[('sweater', 'NN')]
[('fit', 'NN')]
[('her', 'PRP$')]
[('perfectly', 'RB')]
```

In [11]:
```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
documentA = 'Jupiter is the largest Planet'
documentB = 'Mars is the fourth planet from the Sun'

bagOfWordsA = documentA.split(' ')
bagOfWordsB = documentB.split(' ')

uniqueWords = set(bagOfWordsA).union(set(bagOfWordsB))
```

In [12]:
```
numOfWordsA = dict.fromkeys(uniqueWords, 0)
for word in bagOfWordsA:
      numOfWordsA[word] += 1
      numOfWordsB = dict.fromkeys(uniqueWords, 0)
for word in bagOfWordsB:
      numOfWordsB[word] += 1
```

In [15]:
```
def computeTF(wordDict, bagOfWords):
    tfDict = {}
    bagOfWordsCount = len(bagOfWords)
    for word, count in wordDict.items():
           tfDict[word] = count / float(bagOfWordsCount)
    return tfDict
tfA = computeTF(numOfWordsA, bagOfWordsA)
tfB = computeTF(numOfWordsB, bagOfWordsB)
```

In [24]:
```
def computeIDF(documents):
    import math
    N = len(documents)

    idfDict = dict.fromkeys(documents[0].keys(), 0)
    for document in documents:
        for word, val in document.items():
            if val > 0:
                idfDict[word] += 1

    for word, val in idfDict.items():
          idfDict[word] = math.log(N / float(val), 10)
    return idfDict
```

In [23]:
```
idfs = computeIDF([numOfWordsA, numOfWordsB])
def computeTFIDF(tfBagOfWords, idfs):
     tfidf = {}
     for word, val in tfBagOfWords.items():
           tfidf[word] = val * idfs[word]
     return tfidf
tfidfA = computeTFIDF(tfA, idfs)
tfidfB = computeTFIDF(tfB, idfs)
df = pd.DataFrame([tfidfA, tfidfB])
df
```

Out[23]:

	fourth	Planet	Mars	from	is	Sun	the	planet	Jupiter	largest
0	0.000000	0.2	0.000	0.000	0.40	0.000	0.4	0.000	0.2	0.2
1	0.037629	0.0	0.125	0.125	0.25	0.125	0.5	0.125	0.0	0.0

In [ ]: